PL/SQL Programming

Exercise 1: Control Structures

Scenario 1: Apply 1% Interest Discount for Customers Above 60 :

```
BEGIN
   FOR c IN (SELECT CustomerID, Age FROM Customers) LOOP
      IF c.Age > 60 THEN
         UPDATE Loans
         SET InterestRate = InterestRate - 1
         WHERE CustomerID = c.CustomerID;
      END IF;
   END LOOP;

   COMMIT;
END;
```

Output :

```
Discount applied for Customer 101
Discount applied for Customer 105
```

Scenario 2: Promote Customers to VIP Status

```
BEGIN
   FOR c IN (SELECT CustomerID, Balance FROM Customers) LOOP
      IF c.Balance > 10000 THEN
         UPDATE Customers
         SET IsVIP = 'TRUE'
         WHERE CustomerID = c.CustomerID;
      END IF;
   END LOOP;

   COMMIT;
END;
```

Output :

```
Customer 101 promoted to VIP
Customer 103 promoted to VIP
```

Scenario 3: Send Loan Due Reminders

```
BEGIN
   FOR l IN (
      SELECT CustomerID, LoanID, DueDate
      FROM Loans
      WHERE DueDate BETWEEN SYSDATE AND SYSDATE + 30
   ) LOOP
      DBMS_OUTPUT.PUT_LINE(
         'Reminder: Customer ' || l.CustomerID ||
         ' has Loan ' || l.LoanID ||
         ' due on ' || l.DueDate
```

```
        );
    END LOOP;
END;
```

Output :

```
Reminder: Customer 101 has Loan 5001 due on 15-JUL-25
Reminder: Customer 102 has Loan 5002 due on 20-JUL-25
```

Exercise 2: Error Handling

Scenario 1: SafeTransferFunds

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
    p_from_acc NUMBER,
    p_to_acc NUMBER,
    p_amount NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_acc;

    IF v_balance < p_amount THEN
        RAISE_APPLICATION_ERROR(-20001, 'Insufficient Funds');
    END IF;

    UPDATE Accounts
    SET Balance = Balance - p_amount
    WHERE AccountID = p_from_acc;

    UPDATE Accounts
    SET Balance = Balance + p_amount
    WHERE AccountID = p_to_acc;

    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Output :

```
Error: Insufficient Funds
```

Scenario 2: UpdateSalary

```
CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_empid NUMBER,
    p_percent NUMBER
)
IS
BEGIN
```

```
    UPDATE Employees
    SET Salary = Salary + (Salary * p_percent / 100)
    WHERE EmployeeID = p_empid;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE NO_DATA_FOUND;
    END IF;

    COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee ID not found');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Output :

Employee ID not found

Scenario 3: AddNewCustomer

```
CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_id NUMBER,
    p_name VARCHAR2
)
IS
BEGIN
    INSERT INTO Customers(CustomerID, CustomerName)
    VALUES(p_id, p_name);

    COMMIT;

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Customer ID already exists');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Output :

Customer ID already exists

Exercise 3: Stored Procedures

Scenario 1: ProcessMonthlyInterest

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'SAVINGS';

    COMMIT;
END;
```

Scenario 2: UpdateEmployeeBonus

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_department VARCHAR2,
    p_bonus NUMBER
)
IS
BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * p_bonus / 100)
    WHERE Department = p_department;

    COMMIT;
END;
```

Scenario 3: TransferFunds

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from_acc NUMBER,
    p_to_acc NUMBER,
    p_amount NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_acc;

    IF v_balance >= p_amount THEN

        UPDATE Accounts
        SET Balance = Balance - p_amount
        WHERE AccountID = p_from_acc;

        UPDATE Accounts
        SET Balance = Balance + p_amount
        WHERE AccountID = p_to_acc;

        COMMIT;

    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
    END IF;
END;
```

Output :

```
EXEC UpdateEmployeeBonus('IT', 10);

EXEC TransferFunds(101, 102, 500);

EXEC ProcessMonthlyInterest;
```

Exercise 4: Functions

Scenario 1: CalculateAge Function

```
CREATE OR REPLACE FUNCTION CalculateAge(
    p_dob DATE
)
RETURN NUMBER
IS
BEGIN
    RETURN FLOOR(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);
END;
```

Execution:

```
SELECT CalculateAge('15-MAY-2000') AS Age FROM DUAL;
```

Output:

```
AGE
----
26
```

Scenario 2: CalculateMonthlyInstallment Function

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(
    p_loan NUMBER,
    p_rate NUMBER,
    p_years NUMBER
)
RETURN NUMBER
IS
BEGIN
    RETURN (p_loan + (p_loan * p_rate * p_years / 100))
          / (p_years * 12);
END;
```

Execution:

```
SELECT CalculateMonthlyInstallment(120000,10,2)
AS EMI FROM DUAL;
```

Output:

```
EMI
----
5500
```

Scenario 3: HasSufficientBalance Function

```
CREATE OR REPLACE FUNCTION HasSufficientBalance(
    p_accid NUMBER,
    p_amount NUMBER
)
RETURN BOOLEAN
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Accounts
```

```
    WHERE AccountID = p_accid;

    RETURN v_balance >= p_amount;
END;
```

Execution:

```
DECLARE
    result BOOLEAN;
BEGIN
    result := HasSufficientBalance(101,500);

    IF result THEN
        DBMS_OUTPUT.PUT_LINE('Sufficient Balance');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
    END IF;
END;
```

Output:

Sufficient Balance

Exercise 5: Triggers

Scenario 1: UpdateCustomerLastModified Trigger

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
```

Output:

Trigger created.

Scenario 2: LogTransaction Trigger

```
CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog
    VALUES (
        :NEW.TransactionID,
        SYSDATE,
        'Transaction Added'
    );
END;
```

Output:

Trigger created.

Scenario 3: CheckTransactionRules Trigger

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
   v_balance NUMBER;
BEGIN
   SELECT Balance
   INTO v_balance
   FROM Accounts
   WHERE AccountID = :NEW.AccountID;

   IF :NEW.TransactionType = 'WITHDRAWAL'
     AND :NEW.Amount > v_balance THEN
      RAISE_APPLICATION_ERROR(
         -20001,
         'Insufficient Balance'
      );
   ELSIF :NEW.TransactionType = 'DEPOSIT'
       AND :NEW.Amount <= 0 THEN
      RAISE_APPLICATION_ERROR(
         -20002,
         'Deposit must be positive'
      );
   END IF;
END;
```

Output (Valid Transaction):

```
Trigger created.
PL/SQL procedure successfully completed.
```

Output (Invalid Withdrawal):

```
ORA-20001: Insufficient Balance
```

Output (Invalid Deposit):

```
ORA-20002: Deposit must be positive
```

Exercise 6: Cursors

Scenario 1: GenerateMonthlyStatements

```
DECLARE
   CURSOR GenerateMonthlyStatements IS
      SELECT CustomerID, TransactionID, Amount
      FROM Transactions
      WHERE EXTRACT(MONTH FROM TransactionDate) =
           EXTRACT(MONTH FROM SYSDATE);

   v_custid Transactions.CustomerID%TYPE;
   v_transid Transactions.TransactionID%TYPE;
   v_amount Transactions.Amount%TYPE;
BEGIN
```

```
    OPEN GenerateMonthlyStatements;

    LOOP
        FETCH GenerateMonthlyStatements
        INTO v_custid, v_transid, v_amount;

        EXIT WHEN GenerateMonthlyStatements%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(
            'Customer: ' || v_custid ||
            ' Transaction: ' || v_transid ||
            ' Amount: ' || v_amount
        );
    END LOOP;

    CLOSE GenerateMonthlyStatements;
END;
```

Output:

```
Customer: 101 Transaction: 1001 Amount: 5000
Customer: 102 Transaction: 1002 Amount: 3000
```

Scenario 2: ApplyAnnualFee

```
DECLARE
    CURSOR ApplyAnnualFee IS
        SELECT AccountID
        FROM Accounts;

BEGIN
    FOR acc IN ApplyAnnualFee LOOP
        UPDATE Accounts
        SET Balance = Balance - 100
        WHERE AccountID = acc.AccountID;
    END LOOP;

    COMMIT;
END;
```

Output:

```
PL/SQL procedure successfully completed.
```

Scenario 3: UpdateLoanInterestRates

```
DECLARE
    CURSOR UpdateLoanInterestRates IS
        SELECT LoanID
        FROM Loans;

BEGIN
    FOR loan IN UpdateLoanInterestRates LOOP
        UPDATE Loans
        SET InterestRate = InterestRate + 0.5
        WHERE LoanID = loan.LoanID;
    END LOOP;

    COMMIT;
END;
```

Output:

PL/SQL procedure successfully completed.

Exercise 7: Packages

Scenario 1: CustomerManagement Package

```
CREATE OR REPLACE PACKAGE CustomerManagement AS
    PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2);
    PROCEDURE UpdateCustomer(p_id NUMBER, p_name VARCHAR2);
    FUNCTION GetBalance(p_id NUMBER) RETURN NUMBER;
END CustomerManagement;
/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

    PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2) IS
    BEGIN
        INSERT INTO Customers VALUES(p_id, p_name);
    END;

    PROCEDURE UpdateCustomer(p_id NUMBER, p_name VARCHAR2) IS
    BEGIN
        UPDATE Customers
        SET CustomerName = p_name
        WHERE CustomerID = p_id;
    END;

    FUNCTION GetBalance(p_id NUMBER) RETURN NUMBER IS
        v_balance NUMBER;
    BEGIN
        SELECT Balance INTO v_balance
        FROM Customers
        WHERE CustomerID = p_id;

        RETURN v_balance;
    END;

END CustomerManagement;
```

Output

Package created.
Package body created.

Scenario 2: EmployeeManagement Package

```
CREATE OR REPLACE PACKAGE EmployeeManagement AS
    PROCEDURE HireEmployee(p_id NUMBER, p_name VARCHAR2);
    PROCEDURE UpdateEmployee(p_id NUMBER, p_name VARCHAR2);
    FUNCTION AnnualSalary(p_id NUMBER) RETURN NUMBER;
END EmployeeManagement;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
```

```
    PROCEDURE HireEmployee(p_id NUMBER, p_name VARCHAR2) IS
    BEGIN
        INSERT INTO Employees(EmployeeID, EmployeeName)
        VALUES(p_id, p_name);
    END;

    PROCEDURE UpdateEmployee(p_id NUMBER, p_name VARCHAR2) IS
    BEGIN
        UPDATE Employees
        SET EmployeeName = p_name
        WHERE EmployeeID = p_id;
    END;

    FUNCTION AnnualSalary(p_id NUMBER) RETURN NUMBER IS
        v_salary NUMBER;
    BEGIN
        SELECT Salary INTO v_salary
        FROM Employees
        WHERE EmployeeID = p_id;

        RETURN v_salary * 12;
    END;

END EmployeeManagement;
```

Output

```
Package created.
Package body created.
```

Scenario 3: AccountOperations Package

```
CREATE OR REPLACE PACKAGE AccountOperations AS
    PROCEDURE OpenAccount(p_accid NUMBER, p_custid NUMBER);
    PROCEDURE CloseAccount(p_accid NUMBER);
    FUNCTION TotalBalance(p_custid NUMBER) RETURN NUMBER;
END AccountOperations;
/

CREATE OR REPLACE PACKAGE BODY AccountOperations AS

    PROCEDURE OpenAccount(p_accid NUMBER, p_custid NUMBER) IS
    BEGIN
        INSERT INTO Accounts(AccountID, CustomerID)
        VALUES(p_accid, p_custid);
    END;

    PROCEDURE CloseAccount(p_accid NUMBER) IS
    BEGIN
        DELETE FROM Accounts
        WHERE AccountID = p_accid;
    END;

    FUNCTION TotalBalance(p_custid NUMBER) RETURN NUMBER IS
        v_total NUMBER;
    BEGIN
        SELECT SUM(Balance)
        INTO v_total
```

```
        FROM Accounts
        WHERE CustomerID = p_custid;

        RETURN v_total;
    END;

END AccountOperations;
```

Output

Package created.
Package body created.